

Pandas — LEVEL 19: Reading Files

Goal: Learn CSV, Excel, JSON, HTML, file paths, and the important parameters used when reading files.

1. CSV — `pd.read_csv()`

CSV means Comma-Separated Values. It is one of the most common data-file formats.

```
import pandas as pd

df = pd.read_csv("sales.csv")
print(df)
```

Output:

	Name	Age	Sales
0	Asha	21	5000
1	Ravi	25	7000
2	Neha	23	6000

Line by line: 1) import pandas. 2) read the CSV into a DataFrame. 3) print the DataFrame.

2. Excel — `pd.read_excel()`

```
df = pd.read_excel("sales.xlsx")
print(df)
```

Output:

	Name	Age	Sales
0	Asha	21	5000
1	Ravi	25	7000
2	Neha	23	6000

Read a specific worksheet:

```
df = pd.read_excel("sales.xlsx", sheet_name="January")
print(df)
```

3. JSON — `pd.read_json()`

```
df = pd.read_json("sales.json")
print(df)
```

Example JSON:

```
[
  {"Name": "Asha", "Age": 21, "Sales": 5000},
  {"Name": "Ravi", "Age": 25, "Sales": 7000},
  {"Name": "Neha", "Age": 23, "Sales": 6000}
]
```

Output:

	Name	Age	Sales
0	Asha	21	5000
1	Ravi	25	7000
2	Neha	23	6000

4. HTML — `pd.read_html()`

`read_html()` extracts HTML tables and normally returns a list of DataFrames.

```
tables = pd.read_html("sales.html")
df = tables[0]
print(df)
```

Output:

	Name	Age	Sales
0	Asha	21	5000
1	Ravi	25	7000
2	Neha	23	6000

5. File Paths

Relative path: starts from the current working directory.

```
df = pd.read_csv("data/sales.csv")
```

Absolute path: gives the complete location.

```
df = pd.read_csv(r"C:\Users\Jenish\Documents\data\sales.csv")
```

On Windows, a raw string (r"(...)") or forward slashes can make paths easier to write.

6. Delimiters — sep

A delimiter separates values. Comma is common, but files may use semicolons or tabs.

```
df = pd.read_csv("employees.csv", sep=";")  
print(df)
```

For tab-separated data:

```
df = pd.read_csv("employees.tsv", sep="\t")
```

7. Headers — header and names

Use the first row as the header:

```
df = pd.read_csv("sales.csv", header=0)
```

If the file has no header:

```
df = pd.read_csv("sales.csv", header=None)  
print(df)
```

Give your own column names:

```
df = pd.read_csv(  
    "sales.csv",  
    header=None,  
    names=["Name", "Age", "Sales"]  
)  
print(df)
```

8. usecols

usecols reads only selected columns.

```
df = pd.read_csv(  
    "sales.csv",  
    usecols=["Name", "Sales"]  
)  
print(df)
```

Output:

	Name	Sales
0	Asha	5000
1	Ravi	7000
2	Neha	6000

9. skiprows

skiprows skips rows while reading.

```
df = pd.read_csv("sales.csv", skiprows=2)
print(df)
```

Specific rows can also be skipped:

```
df = pd.read_csv("sales.csv", skiprows=[0, 2])
```

Be careful: skipping rows can also change which row becomes the header.

10. nrows

nrows limits how many rows are read.

```
df = pd.read_csv("sales.csv", nrows=2)
print(df)
```

Output:

	Name	Age	Sales
0	Asha	21	5000
1	Ravi	25	7000

11. index_col

index_col makes a selected column the DataFrame index.

```
df = pd.read_csv("sales.csv", index_col="Name")
print(df)
```

Output:

	Age	Sales
Asha	21	5000
Ravi	25	7000
Neha	23	6000

12. dtype

dtype specifies the data type while reading.

```
df = pd.read_csv(
    "sales.csv",
    dtype={"Age": "int64", "Sales": "float64"}
)
print(df.dtypes)
```

Output:

Name	object
Age	int64
Sales	float64
dtype:	object

For IDs where leading zeros matter:

```
df = pd.read_csv("employees.csv", dtype={"EmpID": "string"})
```

13. parse_dates

parse_dates converts selected columns into datetime values.

```
df = pd.read_csv(
    "sales.csv",
    parse_dates=["Date"]
)
print(df.dtypes)
```

Output:

```
Date      datetime64[ns]
Name      object
Sales     int64
dtype: object
```

Then date components can be accessed with .dt, for example:

```
print(df["Date"].dt.year)
```

14. Encoding

Encoding controls how text bytes are decoded. UTF-8 is common.

```
df = pd.read_csv(
    "customers.csv",
    encoding="utf-8"
)
```

Another encoding sometimes encountered is latin1:

```
df = pd.read_csv(
    "customers.csv",
    encoding="latin1"
)
```

If you get UnicodeDecodeError, check the file's actual encoding instead of guessing.

15. Combine Parameters

```
df = pd.read_csv(
    "sales.csv",
    sep=",",
    header=0,
    usecols=["Date", "Name", "Sales"],
    nrows=5,
    dtype={"Name": "string"},
    parse_dates=["Date"],
    encoding="utf-8"
)

print(df)
print(df.dtypes)
```

Output:

```
Date  Name  Sales
0 2026-01-01  Asha   5000
1 2026-01-02   Ravi   7000
2 2026-01-03  Neha   6000
3 2026-01-04   Amit   8000
4 2026-01-05   Isha   4500
```

```
Date      datetime64[ns]
Name      string
Sales     int64
dtype: object
```

16. Quick Revision

```
CSV      -> pd.read_csv()
Excel    -> pd.read_excel()
JSON     -> pd.read_json()
HTML     -> pd.read_html()
```

```
Relative path -> from current folder
Absolute path -> complete location
```

sep	-> delimiter
header	-> header row
usecols	-> selected columns
skiprows	-> skipped rows
nrows	-> number of rows to read
index_col	-> index column
dtype	-> data type
parse_dates	-> datetime conversion
encoding	-> text decoding

17. Common Errors

FileNotFoundError: check the filename and path.

UnicodeDecodeError: check encoding.

ParserError: check delimiter and malformed rows.

KeyError: check column names used in usecols.

Excel engine error: check that the required Excel-reading dependency is installed.

18. Interview Questions

1. What is the difference between relative and absolute paths?
2. What does usecols do?
3. Why use nrows?
4. What does index_col do?
5. Why use parse_dates?
6. What does dtype do?
7. What is a delimiter?
8. What does read_html() return?
9. How do you read a semicolon-separated file?
10. How do you handle an encoding error?

Best practice: First test a file with nrows, inspect df.head() and df.dtypes, then load the full dataset after the settings are correct.